Ch4.1: Chẩn đoán trạng thái ổ lăn bằng mạng MLP

Bài toán **phân loại đa lớp**: nhận dạng 10 trạng thái của ổ lăn từ tín hiệu rung của bộ dữ liệu CWRU (mức tải 0 HP, cảm biến đầu trục, 12.000 mẫu/s).

Tín hiệu mỗi trạng thái được cắt thành 5 đoạn thời gian liên tiếp: 3 đoạn đầu để huấn luyện, đoạn thứ tư để kiểm định, đoạn cuối để kiểm tra.

Mỗi mạng được huấn luyện lại với ba hạt giống (16, 5, 20) và báo cáo bằng trung bình ± độ lệch chuẩn. Trong thực tế nên chạy nhiều hạt giống hơn (thường từ 5 đến 10) để đánh giá đúng độ ổn định của kết quả; ở đây là bài tập nên chỉ dùng 3 hạt giống để tiết kiệm thời gian chạy.

Chuẩn bị dữ liệu. Tải mười tệp .mat từ https://engineering.case.edu/bearingdatacenter/download-data-file rồi đặt vào thư mục `cwru` trên Google Drive.

Mục	Nội dung
1	Chuẩn bị
2	Xác định bài toán, dữ liệu
3	Chia tập và tiền xử lý
4	Khai báo và huấn luyện mạng cơ bản
5	Cải tiến theo phương pháp thử và sai (Trial and Error)
6	Tinh chỉnh siêu tham số
7	Lưu và dùng lại mô hình
8	Tổng kết quy trình

1. Chuẩn bị

Nạp thư viện và thiết lập hình vẽ

Ô này nạp các thư viện dùng chung: `numpy` (tính toán mảng), `matplotlib` (vẽ hình) và `tensorflow` (xây dựng mạng nơ-ron), rồi in phiên bản TensorFlow đang chạy. `plt.rcParams` đặt font, độ phân giải và bỏ khung trên, khung phải cho mọi hình; từ điển `COLORS` giữ bảng màu thống nhất cho các hình ở các mục sau.

In [ ]:

```
import os, time
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf

print("TensorFlow", tf.__version__)
```

```
plt.rcParams.update({"font.family": "DejaVu Sans", "font.size": 11,
                     "figure.dpi": 150, "axes.spines.top": False,
                     "axes.spines.right": False})
COLORS = {"grey": "#6E6D69", "purple": "#4A3AA7", "yellow": "#EDA100",
          "blue": "#2A78D6", "orange": "#EB6834", "red": "#E34948",
          "green": "#1BAF7A", "ink": "#0B0B0B"}
```

```
TensorFlow 2.21.0
```

2. Xác định bài toán, dữ liệu

2.1. Đọc dữ liệu rung CWRU

Ô này kết nối Google Drive (`drive.mount`) và đọc mười tệp `.mat` của bộ dữ liệu CWRU, mỗi tệp ứng với một trạng thái ổ lăn ở mức tải 0 HP. Từ điển `FILE_IDS` ánh xạ tên trạng thái với mã tệp; hàm `load_drive_end_signal` dùng `scipy.io.loadmat` lấy tín hiệu gia tốc ở đầu trục (khoá có đuôi `DE_time`), lấy mẫu 12 kHz. Kết quả: mười tín hiệu, mỗi trạng thái hỏng khoảng 121–123 nghìn mẫu, riêng trạng thái bình thường có 243.938 mẫu, gấp đôi.

In [ ]:

```
import numpy as np
from scipy.io import loadmat
from google.colab import drive

drive.mount("/content/gdrive")      # kết nối Google Drive
DATA_DIR = "/content/gdrive/MyDrive/Colab Notebooks/Data/cwru"  # Khai báo đúng đường dẫn thư mục

# Mười trạng thái, mức tải 0 HP, tín hiệu đầu trục (DE), 12 kHz
FILE_IDS = {
    "Bình thường":         "97",
    "Vòng trong 0,007in": "105", "Vòng trong 0,014in": "169",
    "Vòng trong 0,021in": "209",
    "Con lăn 0,007in":    "118", "Con lăn 0,014in":    "185",
    "Con lăn 0,021in":    "222",
    "Vòng ngoài 0,007in": "130", "Vòng ngoài 0,014in": "197",
    "Vòng ngoài 0,021in": "234",
}

def load_drive_end_signal(file_id):
    """Trả về chuỗi gia tốc ở đầu trục trong tệp .mat."""
    mat = loadmat(f"{DATA_DIR}/{file_id}.mat")
    key = [name for name in mat if name.endswith("DE_time")][0]
    return mat[key].ravel()

signals = {state: load_drive_end_signal(file_id)
           for state, file_id in FILE_IDS.items()}
for state, samples in signals.items():
    print("%-20s %7d mẫu" % (state, len(samples)))
```

```
Bình thường            243938 mẫu
Vòng trong 0,007in      121265 mẫu
Vòng trong 0,014in      121846 mẫu
Vòng trong 0,021in      122136 mẫu
Con lăn 0,007in         122571 mẫu
Con lăn 0,014in         121846 mẫu
Con lăn 0,021in         121991 mẫu
Vòng ngoài 0,007in      121991 mẫu
Vòng ngoài 0,014in      121846 mẫu
Vòng ngoài 0,021in      122426 mẫu
```

2.2. Quan sát dạng sóng

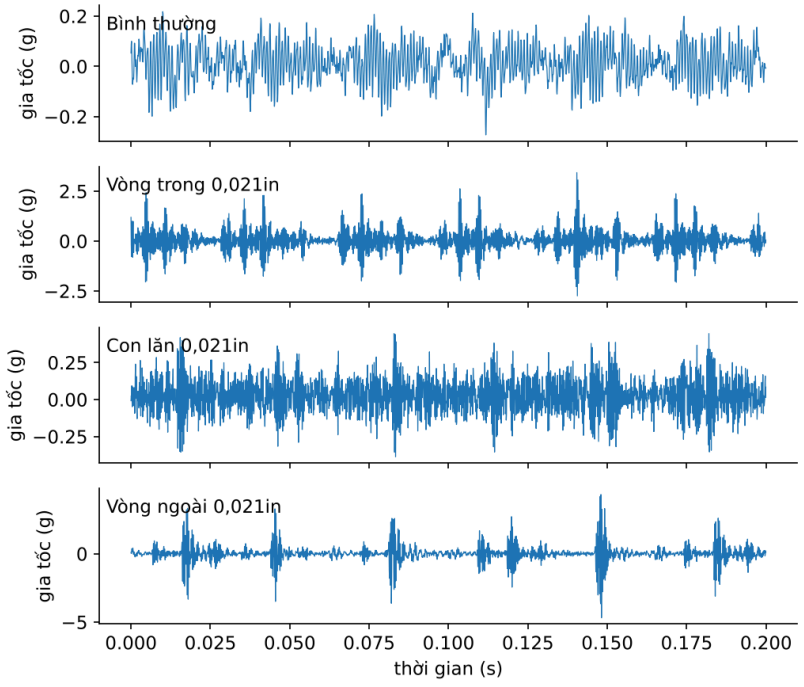
Ô này vẽ 0,2 giây đầu (2400 mẫu) của bốn trạng thái: bình thường, vòng trong, con lăn và vòng ngoài, cùng cỡ hư hỏng 0,021 in, bằng `plt.subplots` với trục thời gian chung. Kết quả: ổ hỏng vòng trong và vòng ngoài có các xung va đập đều đặn với biên độ cỡ 2,5 g và 4 g, gấp hàng chục lần trạng thái bình thường (khoảng 0,2 g); tín hiệu con lăn có biên độ gần như trạng thái bình thường nên khó phân biệt bằng mắt, vì vậy cần trích đặc trưng ở mục sau.

In [ ]:

```
import matplotlib.pyplot as plt

SELECTED_STATES = ["Bình thường", "Vòng trong 0,021in",
                   "Con lăn 0,021in", "Vòng ngoài 0,021in"]
SAMPLE_RATE = 12000            # tần số lấy mẫu, Hz
PLOT_SAMPLES = 2400            # 0,2 giây đầu của mỗi tín hiệu
time_axis = np.arange(PLOT_SAMPLES) / SAMPLE_RATE

fig, axes = plt.subplots(4, 1, figsize=(7, 6), sharex=True)
for ax, state in zip(axes, SELECTED_STATES):
    ax.plot(time_axis, signals[state][:PLOT_SAMPLES], lw=0.6)
    ax.set_ylabel("gia tốc (g)")
    ax.text(0.01, 0.82, state, transform=ax.transAxes)
axes[-1].set_xlabel("thời gian (s)")
plt.tight_layout()
plt.show()
```



3. Chia tập và tiền xử lý

3.1. Trích chín đặc trưng cho mỗi cửa sổ

Ô này định nghĩa cách biến một cửa sổ tín hiệu 2048 điểm (bước trượt 1024 điểm) thành chín đặc trưng. Hàm `extract_features` tính sáu đặc trưng thống kê miền thời gian: RMS, giá trị đỉnh, hệ số đỉnh, độ nhọn, độ lệch và hệ số xung. Ba đặc trưng còn lại là biên độ phổ bao tại tần số hư hỏng vòng trong (BPFI), vòng ngoài (BPFO) và con lăn (BSF), do hàm `envelope_peak` tính bằng biến đổi Hilbert, cửa sổ Hann và FFT; các tần số này suy ra từ tốc độ quay trục `SHAFT_FREQ` = 1797/60 Hz. Ô in ra danh sách chín đặc trưng.

In [ ]:

```
from scipy.signal import hilbert
from scipy.stats import kurtosis, skew

SAMPLE_RATE = 12000          # tần số lấy mẫu, Hz
WINDOW_SIZE = 2048           # độ dài một cửa sổ tín hiệu
STRIDE      = 1024           # bước trượt giữa hai cửa sổ liên tiếp

SHAFT_FREQ = 1797 / 60             # tần số quay trục tại 0 HP, Hz
FAULT_HZ = {"BPFI": 5.4152 * SHAFT_FREQ,   # hư hỏng vòng bi trong
            "BPFO": 3.5848 * SHAFT_FREQ,   # hư hỏng vòng bi ngoài
            "BSF": 4.7135 * SHAFT_FREQ}    # hư hỏng con lăn

def envelope_peak(x, freq, band=3.0):
    """Biên độ lớn nhất của phổ bao quanh freq, ±band Hz."""
    envelope = np.abs(hilbert(x))          # đường bao Hilbert
    envelope = envelope - envelope.mean()    # bỏ thành phần 0 Hz
    window = np.hanning(len(envelope))
    spectrum = np.abs(np.fft.rfft(envelope * window))
    spectrum = 2 * spectrum / len(envelope)
    freqs = np.fft.rfftfreq(len(envelope), 1 / SAMPLE_RATE)
    in_band = (freqs > freq - band) & (freqs < freq + band)
    return spectrum[in_band].max()

def extract_features(x):
    rms      = np.sqrt(np.mean(x ** 2))
    peak     = np.max(np.abs(x))
    mean_abs = np.mean(np.abs(x))
    return [rms,                                  # 1. RMS
            peak,                                 # 2. giá trị đỉnh
            peak / rms,                           # 3. hệ số đỉnh
            kurtosis(x),                          # 4. độ nhọn
            skew(x),                              # 5. độ lệch
            peak / mean_abs,                      # 6. hệ số xung
            envelope_peak(x, FAULT_HZ["BPFI"]),   # 7. phổ bao BPFI
            envelope_peak(x, FAULT_HZ["BPFO"]),   # 8. phổ bao BPFO
            envelope_peak(x, FAULT_HZ["BSF"])]    # 9. phổ bao BSF

# Danh sách 9 đặc trưng
feature_names = [
    "1. RMS",
    "2. Giá trị đỉnh",
    "3. Hệ số đỉnh",
    "4. Độ nhọn",
```

```
    "5. Độ lệch",
    "6. Hệ số xung",
    "7. Phổ bao BPFI",
    "8. Phổ bao BPFO",
    "9. Phổ bao BSF"
]

print("Danh sách 9 đặc trưng:")
for feature in feature_names:
    print(feature)
```

```
Danh sách 9 đặc trưng:
1. RMS
2. Giá trị đỉnh
3. Hệ số đỉnh
4. Độ nhọn
5. Độ lệch
6. Hệ số xung
7. Phổ bao BPFI
8. Phổ bao BPFO
9. Phổ bao BSF
```

3.2. Cắt đoạn thời gian và dựng bảng đặc trưng

Ô này cắt mỗi tín hiệu thành 5 đoạn liên tiếp, rồi lấy các cửa sổ nằm trọn trong từng đoạn, không vắt qua ranh giới. Với mỗi cửa sổ, ô lưu chín đặc trưng vào `X`, nhãn trạng thái (0–9) vào `y` và chỉ số đoạn chứa cửa sổ vào `segment`; chỉ số đoạn dùng để chia tập ở ô sau. `pandas.DataFrame` gom kết quả thành bảng. Kết quả: 1100 cửa sổ (110 cửa sổ cho mỗi trạng thái), bảng có 9 cột đặc trưng và 1 cột nhãn; ô hiển thị 10 dòng ngẫu nhiên.

In [ ]:

```
import pandas as pd

NUM_SEGMENTS = 5      # mỗi tín hiệu cắt thành 5 đoạn liên tiếp
class_names = list(FILE_IDS)
min_length = min(len(x) for x in signals.values())

X, y, segment = [], [], []
for label, state in enumerate(class_names):
    x = signals[state][:min_length]    # cùng độ dài mọi trạng thái
    edges = np.linspace(0, min_length,
                        NUM_SEGMENTS + 1).astype(int)
    for index in range(NUM_SEGMENTS):
        # cửa sổ nằm trọn trong một đoạn, không vắt qua ranh giới
        last_start = edges[index + 1] - WINDOW_SIZE
        for start in range(edges[index], last_start + 1, STRIDE):
            window = x[start:start + WINDOW_SIZE]
            X.append(extract_features(window))
            y.append(label)
            segment.append(index)      # đoạn chứa cửa sổ này
X = np.asarray(X, dtype="float32")
y, segment = np.asarray(y), np.asarray(segment)

# Tạo DataFrame chứa 9 đặc trưng và 1 cột nhãn
df = pd.DataFrame(X, columns=feature_names)
df['Nhãn (Label)'] = y

# Hiển thị 10 dòng ngẫu nhiên từ bảng dữ liệu
print("Bảng dữ liệu:")
```

```
display(df.sample(10, random_state=16))
print("Kích thước bảng dữ liệu:", df.shape)
```

Bảng dữ liệu:

	1. RMS	2. Giá trị đỉnh	3. Hệ số đỉnh	4. Độ nhọn	5. Độ lệch	6. Hệ số xung	7. Phổ bao BPFI	8. Phổ bao BPFO	9. Phổ ba BS
413	0.544160	3.556923	6.536537	5.182590	0.335820	9.658978	0.150592	0.012581	0.01400
759	0.116772	0.533112	4.565392	1.040080	-0.014054	5.878983	0.011649	0.010744	0.00537
171	0.290989	1.309877	4.501471	1.898762	0.150733	6.217502	0.092613	0.035590	0.01163
30	0.072139	0.220298	3.053805	-0.350671	0.004328	3.758699	0.000463	0.001850	0.00167
252	0.201762	1.655864	8.207006	18.005360	0.081985	13.648228	0.042739	0.029243	0.01453
810	0.684217	3.347788	4.892876	4.615215	0.081136	8.131516	0.025261	0.314782	0.02187
7	0.073146	0.247418	3.382503	0.019777	-0.132286	4.253822	0.001470	0.002718	0.00053
613	0.103701	0.659649	6.361095	2.621979	0.123857	8.551091	0.003298	0.002830	0.00494
961	0.100754	0.387150	3.842525	0.081852	-0.084416	4.840364	0.006041	0.004376	0.00343
627	0.140793	1.391744	9.885024	14.235558	0.440695	14.668927	0.001720	0.006509	0.00595

```
Kích thước bảng dữ liệu: (1100, 10)
```

3.3. Chia tập huấn luyện, kiểm định và kiểm tra

Ô này chia dữ liệu theo đoạn thời gian: ba đoạn đầu để huấn luyện, đoạn thứ tư để kiểm định, đoạn cuối để kiểm tra. Ba tập không có chung khoảng thời gian nào, nên tập kiểm định và kiểm tra không chứa cửa sổ lân cận với cửa sổ đã học, tránh đánh giá lạc quan quá mức. Kết quả: 660 mẫu huấn luyện, 220 mẫu kiểm định và 220 mẫu kiểm tra, mỗi mẫu có 9 đặc trưng.

In [ ]:

```
from sklearn.preprocessing import StandardScaler
from tensorflow import keras

# Ba đoạn đầu để huấn luyện, đoạn 4 để kiểm định, đoạn 5 để
# kiểm tra: ba tập không có chung một khoảng thời gian nào
is_train = segment <= 2
is_val, is_test = segment == 3, segment == 4
X_train, y_train = X[is_train], y[is_train]
X_val, y_val = X[is_val], y[is_val]
X_test, y_test = X[is_test], y[is_test]

print("Train:      ", X_train.shape)
print("Validation: ", X_val.shape)
print("Test:       ", X_test.shape)
```

```
Train:       (660, 9)
Validation:  (220, 9)
Test:        (220, 9)
```

3.4. Chuẩn hoá đặc trưng

Ô này dùng `StandardScaler` đưa mỗi đặc trưng về trung bình 0 và độ lệch chuẩn 1. Tham số chuẩn hoá chỉ ước lượng trên tập huấn luyện rồi áp cho cả ba tập, để thông tin của tập kiểm định và kiểm tra không lọt vào quá trình huấn luyện. Kết quả: bảng 660 dòng huấn luyện sau chuẩn hoá, các đặc trưng có cùng thang đo.

In [ ]:

```
# Tham số chuẩn hoá chỉ ước lượng trên tập huấn luyện
scaler = StandardScaler().fit(X_train)
```

```
# Chuẩn hóa các tập dữ liệu dựa trên tham số chuẩn hóa
X_train = scaler.transform(X_train)
X_val   = scaler.transform(X_val)
X_test  = scaler.transform(X_test)

# Tạo DataFrame cho tập dữ liệu huấn luyện đã được chuẩn hóa để hiển thị
df_scaled = pd.DataFrame(X_train, columns=feature_names)
df_scaled['Nhãn (Label)'] = y_train

print("Bảng dữ liệu huấn luyện sau khi chuẩn hóa:")
display(df_scaled.head(10))
print("Kích thước bảng dữ liệu huấn luyện đã chuẩn hóa:", df_scaled.shape)
```

```
Bảng dữ liệu huấn luyện sau khi chuẩn hóa:
```

	1. RMS	2. Giá trị đỉnh	3. Hệ số đỉnh	4. Độ nhọn	5. Độ lệch	6. Hệ số xung	7. Phổ bao BPFI	8. Phổ bao BPFO	9. Phổ ba BS
0	-0.990662	-0.935757	-0.861911	-0.772415	-1.342064	-0.849166	-0.802490	-0.585190	-0.64586
1	-0.984812	-0.935757	-0.893521	-0.782443	-0.467870	-0.862244	-0.810123	-0.585551	-0.57787
2	-0.963632	-0.965436	-1.294502	-0.792148	-0.826891	-1.097138	-0.812301	-0.577079	-0.54646
3	-0.968864	-0.965436	-1.273678	-0.787324	-1.111856	-1.083766	-0.831952	-0.607212	-0.60684
4	-0.985604	-0.970270	-1.253921	-0.797436	-0.220096	-1.071686	-0.819393	-0.598187	-0.64935
5	-0.991903	-0.950932	-1.018343	-0.791442	-0.549252	-0.942736	-0.794138	-0.578133	-0.62496
6	-0.988642	-0.950932	-1.034680	-0.775716	-1.412246	-0.949109	-0.832442	-0.579518	-0.62912
7	-0.991177	-0.952141	-1.034984	-0.763085	-1.307463	-0.940426	-0.825242	-0.584862	-0.64761
8	-0.996017	-0.966778	-1.169869	-0.795138	-0.971899	-1.030969	-0.844897	-0.589643	-0.61381
9	-1.003561	-0.966778	-1.133832	-0.794078	-0.800309	-1.006138	-0.843296	-0.603042	-0.60773

```
Kích thước bảng dữ liệu huấn luyện đã chuẩn hóa: (660, 10)
```

4. Khai báo và huấn luyện mạng cơ bản

Mục này gộp ba việc đi liền nhau của quy trình: khai báo mạng, khai báo cấu hình huấn luyện và huấn luyện mạng cơ bản để chẩn đoán. Gồm năm mục: 4.1 định nghĩa mạng cơ sở 9–16–10; 4.2 kha báo cấu hình huấn luyện; 4.3 huấn luyện mạng với một hạt giống; 4.4 chẩn đoán bằng đường học và ma trận nhầm lẫn; 4.5 huấn luyện với ba hạt giống để có mốc so sánh cho các mục sau.

4.1. Định nghĩa mạng cơ sở 9–16–10

Ô này dựng mạng cơ sở bằng `keras.Sequential`: lớp vào 9 đặc trưng, một lớp ẩn `Dense` 16 nơ-ron ReLU và lớp ra `Dense` 10 nơ-ron Softmax (một nơ-ron cho mỗi trạng thái). `set_random_seed(16)` cố định khởi tạo. Mạng `model_bs` chưa huấn luyện; mục 4.3 huấn luyện trực tiếp mạng này, còn mục 4.5 dùng nó làm khuôn, mỗi lần huấn luyện tạo bản sao mới bằng `clone_model`. Kết quả từ `model_bs.summary()`: 330 tham số, gồm 160 ở lớp ẩn và 170 ở lớp ra.

```
In [ ]:
```

```
# Định nghĩa mạng cơ sở 9-16-10 (model_bs): mạng chưa huấn luyện, dùng làm khuôn.
keras.utils.set_random_seed(16)
model_bs = keras.Sequential([
    keras.layers.Input(shape=(9,)),
    keras.layers.Dense(16, activation="relu"),      # lớp ẩn
    keras.layers.Dense(10, activation="softmax"),   # 10 lớp
])
model_bs.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 16)	160
dense_1 (Dense)	(None, 10)	170

Total params: 330 (1.29 KB)
Trainable params: 330 (1.29 KB)
Non-trainable params: 0 (0.00 B)

4.2. Cấu hình huấn luyện (khai báo theo trình tự chung)

Ô này khai báo cách huấn luyện bằng `model_bs.compile`: bộ tối ưu Adam với tốc độ học 10^{-3}, hàm mất mát `sparse_categorical_crossentropy` (nhãn là số nguyên 0–9 nên dùng dạng `sparse`) và phép đo Accuracy. Callback `EarlyStopping` theo dõi mất mát kiểm định với patience 20 và khôi phục trọng số tốt nhất. Ô chỉ khai báo, chưa in kết quả.

Đây là khai báo theo trình tự chung của quy trình: cấu hình biên dịch gắn với chính mô hình `model_bs` và `early_stopping` chỉ được dùng khi gọi `fit()`. Trong bài này cấu hình được dùng ngay ở mục 4.3 qua `model_bs.fit(...)`. Từ mục 4.5 trở đi, mạng được tạo mới bằng `clone_model` (bản sao không mang theo thiết lập biên dịch) hoặc `build_network`, nên được biên dịch lại trong `train_once`, `build_network` và `run_variant`; muốn đổi tốc độ học hay patience ở những mục đó phải sửa tại những hàm này, không sửa ở mục 4.2.

In [ ]:

```
# Bộ tối ưu Adam, hàm mất mát sparse categorical cross-entropy, phép đo Accuracy và
dừng sớm.

model_bs.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-3),
    loss="sparse_categorical_crossentropy", # nhãn là số nguyên 0-9 nên dùng dạng
"sparse"
    metrics=["accuracy"],
)

early_stopping = keras.callbacks.EarlyStopping(
    monitor="val_loss", patience=20, restore_best_weights=True)
```

4.3. Huấn luyện mạng cơ sở với một hạt giống

Ô này huấn luyện trực tiếp mạng cơ sở `model_bs` bằng cấu hình đã khai báo ở mục 4.2: `model_bs.fit` với dữ liệu huấn luyện, dữ liệu kiểm định `(X_val, y_val)`, tối đa 300 epoch, lô 32 mẫu và callback `early_stopping`. Hạt giống 16 đã đặt khi định nghĩa mạng ở mục 4.1. Giá trị trả về của `fit` được lấy bằng `history.history` để vẽ đường học ở ô sau, rồi `model_bs.evaluate` tính Accuracy trên hai tập. Kết quả chính: Accuracy huấn luyện 0,964, Accuracy kiểm định 0,936, và mạng chạy đủ 300 epoch mà dừng sớm không kích hoạt. Kết quả này chỉ từ một hạt giống nên chưa đủ để kết luận; mục 4.5 lặp lại việc huấn luyện với nhiều hạt giống.

In [ ]:

```
# Huấn luyện trực tiếp model_bs bằng cấu hình đã khai báo ở mục 4.2
# (hạt giống 16, đã đặt khi định nghĩa mạng ở mục 4.1)
```

```
history = model_bs.fit(
    X_train, y_train, validation_data=(X_val, y_val),
    epochs=300, batch_size=32, verbose=0,
    callbacks=[early_stopping])
history = history.history

print("Accuracy huấn luyện:",
      round(model_bs.evaluate(X_train, y_train, verbose=0)[1], 3))
print("Accuracy kiểm định :",
      round(model_bs.evaluate(X_val, y_val, verbose=0)[1], 3))
print("Số epoch đã chạy   :", len(history["loss"]))
```

```
Accuracy huấn luyện: 0.964
Accuracy kiểm định : 0.936
Số epoch đã chạy   : 300
```

4.4. Chẩn đoán bằng đường học và ma trận nhầm lẫn

Ô này lấy mạng `model_bs` vừa huấn luyện ở mục 4.3 và vẽ hai hình: đường mất mát huấn luyện, kiểm định theo epoch, và ma trận nhầm lẫn trên tập kiểm định (`confusion_matrix`, `ConfusionMatrixDisplay`). Vòng lặp phía dưới phân loại các cặp nhầm thành bốn loại: sai mức đường kính (cùng dạng hỏng), sai dạng hư hỏng, báo sót (ổ hỏng bị báo tốt) và báo nhầm (ổ tốt bị báo hỏng). Kết quả chính: cả hai đường mất mát còn giảm khi dừng ở epoch 300, dấu hiệu chưa khớp; sai mức 6, sai dạng 8, báo sót 0, báo nhầm 0 (số liệu từ một hạt giống nên thay đổi theo từng lần chạy).

In [ ]:

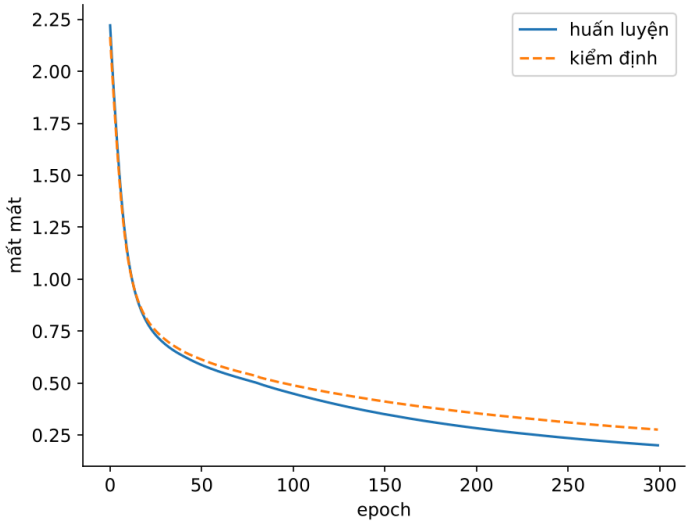
```
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix
from sklearn.metrics import ConfusionMatrixDisplay

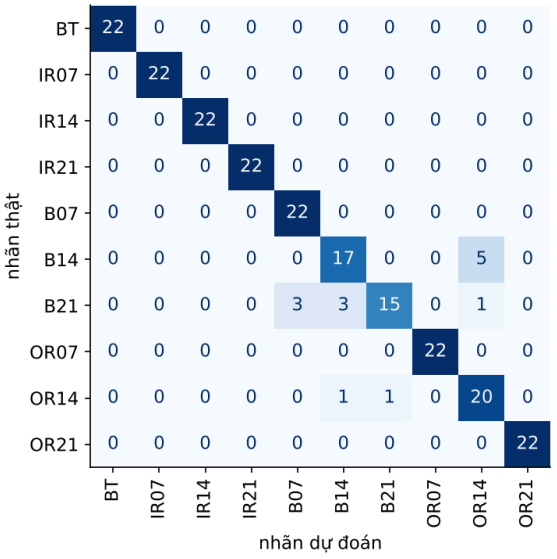
SHORT_LABELS = ["BT", "IR07", "IR14", "IR21", "B07", "B14", "B21",
                "OR07", "OR14", "OR21"]
y_pred = model_bs.predict(X_val, verbose=0).argmax(axis=1)
matrix = confusion_matrix(y_val, y_pred)

fig, (ax_loss, ax_matrix) = plt.subplots(1, 2, figsize=(12, 5))
ax_loss.plot(history["loss"], label="huấn luyện")
ax_loss.plot(history["val_loss"], "--", label="kiểm định")
ax_loss.set_xlabel("epoch"); ax_loss.set_ylabel("mất mát")
ax_loss.legend()
ConfusionMatrixDisplay(matrix, display_labels=SHORT_LABELS).plot(
    ax=ax_matrix, cmap="Blues", colorbar=False,
    xticks_rotation=90)
ax_matrix.set_xlabel("nhãn dự đoán")
ax_matrix.set_ylabel("nhãn thật")
plt.tight_layout()
plt.show()

# Phân loại các cặp nhầm lẫn theo mức độ nghiêm trọng
same_fault = other_fault = missed = false_alarm = 0
for i, j in zip(*np.nonzero(matrix)):
    if i == j:
        continue
    if i == 0:
        false_alarm += matrix[i, j]     # ổ tốt bị báo hỏng
    elif j == 0:
        missed += matrix[i, j]          # ổ hỏng bị báo tốt
```

```
    elif (i - 1) // 3 == (j - 1) // 3:
        same_fault += matrix[i, j]      # sai mức đường kính
    else:
        other_fault += matrix[i, j]     # sai dạng hư hỏng
print("sai mức:", same_fault, "| sai dạng:", other_fault,
      "| báo sót:", missed, "| báo nhầm:", false_alarm)
```






```
sai mức: 6 | sai dạng: 8 | báo sót: 0 | báo nhầm: 0
```

4.5. Huấn luyện mạng cơ sở với ba hạt giống

Ô này huấn luyện lại mạng cơ sở ba lần, mỗi lần một hạt giống (16, 5, 20). Trong thực tế nên chạy nhiều hạt giống hơn (thường từ 5 đến 10) để đánh giá đúng độ ổn định của kết quả; ở đây là bài tập nên giới hạn 3 hạt giống để tiết kiệm thời gian. Hàm `train_once` tạo mạng mới bằng `clone_model(model_bs)`, biên dịch lại với cùng cấu hình ở mục 4.2, huấn luyện tối đa 300 epoch (lô 32 mẫu, dừng sớm), rồi ô tính Accuracy huấn luyện, Accuracy kiểm định và F1 macro (`f1_score`) cho từng hạt giống và gom vào bảng `pandas` kèm trung bình và độ lệch chuẩn. Hạt giống 16 cho lại đúng kết quả của mục 4.3. Kết quả chính: Accuracy huấn luyện 94,4 ± 1,7 %, kiểm định 92,3 ± 1,0 %, và cả ba lần đều chạy đủ 300 epoch mà dừng sớm không kích hoạt, tức mạng chưa học xong. Hạt giống 20 kém nhất (91,4 % trên tập kiểm định), hạt giống 16 tốt nhất (93,6 %): chỉ với ba lần chạy, chênh lệch giữa các hạt giống đã là 2,2 điểm phần trăm.

In [ ]:

```
import statistics
import pandas as pd
from sklearn.metrics import f1_score

SEEDS = (16, 5, 20)    # ba hạt giống; trong thực tế nên dùng nhiều hơn

def train_once(seed):
    keras.utils.set_random_seed(seed)
    # mạng mới có cùng kiến trúc với model_bs, trọng số khởi tạo lại
    model = keras.models.clone_model(model_bs)
    model.compile(optimizer=keras.optimizers.Adam(1e-3),
                  loss="sparse_categorical_crossentropy",
                  metrics=["accuracy"])
    early_stopping = keras.callbacks.EarlyStopping(
        monitor="val_loss", patience=20,
        restore_best_weights=True)
    history = model.fit(
```

```
        X_train, y_train, validation_data=(X_val, y_val),
        epochs=300, batch_size=32, verbose=0,
        callbacks=[early_stopping])
    return model, history.history

runs = [train_once(seed) for seed in SEEDS]
train_acc, val_acc, val_f1, epochs = [], [], [], []
for model, history in runs:
    train_acc.append(model.evaluate(X_train, y_train,
                                    verbose=0)[1])
    val_acc.append(model.evaluate(X_val, y_val, verbose=0)[1])
    y_pred = model.predict(X_val, verbose=0).argmax(axis=1)
    val_f1.append(f1_score(y_val, y_pred, average="macro"))
    epochs.append(len(history["loss"]))

# Bảng kết quả từng hạt giống, kèm trung bình và độ lệch chuẩn
per_seed = pd.DataFrame({
    "Accuracy huấn luyện (%)": [100 * v for v in train_acc],
    "Accuracy kiểm định (%)": [100 * v for v in val_acc],
    "F1 macro kiểm định (%)": [100 * v for v in val_f1],
    "Số epoch": epochs}, index=pd.Index(SEEDS, name="Hạt giống"))
summary = pd.DataFrame({
    "Trung bình": per_seed.mean(),
    "Độ lệch chuẩn (±)": per_seed.apply(statistics.pstdev)}).T
display(pd.concat([per_seed, summary]).round(1))
```

	Accuracy huấn luyện (%)	Accuracy kiểm định (%)	F1 macro kiểm định (%)
16	96.4	93.6	93.5
5	94.7	91.8	91.7
20	92.3	91.4	91.4
Trung bình	94.4	92.3	92.2
Độ lệch chuẩn (±)	1.7	1.0	0.9

5. Cải tiến theo phương pháp thử và sai (Trial and Error)

5.1. Phương pháp thử và sai (Trial and Error) và cách làm ở mục 5

Thử và sai là phương pháp cải tiến mô hình bằng thí nghiệm: mỗi lần chỉ thay đổi một yếu tố, huấn luyện lại, đo kết quả trên tập kiểm định, giữ lại thay đổi có ích và bỏ thay đổi không hiệu quả. Khác với dò tìm ngẫu nhiên ở mục 6, các thử nghiệm ở đây không chọn tuỳ ý mà đi theo chẩn đoán ở mục 4: đường mất mát còn giảm sau 300 epoch cho thấy mạng chưa khớp, nên các phương án đầu tiên nhằm chữa chưa khớp, và sau đó mới xử lý quá khớp phát sinh.

Mục 5 gồm ba nhóm phương án nối tiếp nhau. Mục 5.4 chỉ kéo dài thời gian huấn luyện (tối đa 2000 epoch, để dừng sớm quyết định điểm dừng) và giữ nguyên mạng. Mục 5.5 tăng kích thước mạng lên 9–256–128–10. Mục 5.6 thêm từng kỹ thuật chính quy hoá vào mạng mở rộng (L2, Dropout, dừng sớm, và BatchNormalization để đối chiếu), mỗi lần chỉ thêm một kỹ thuật để thấy tác dụng riêng, rồi gộp cả bốn. Mỗi phương án được huấn luyện với cả ba hạt giống và báo cáo bằng trung bình ± độ lệch chuẩn, vì chênh lệch nhỏ giữa hai phương án có thể chỉ do khởi tạo ngẫu nhiên. Mọi so sánh đều dùng tập kiểm định; tập kiểm tra được để dành cho đánh giá cuối ở mục 6. Cuối mục có bảng tổng hợp tám phương án để chọn hướng cải tiến.

5.2. Hàm `build_network`: dựng mạng có công tắc cho từng kỹ thuật

Hàm `build_network` dựng mạng MLP 9–hidden–10 dùng chung cho mọi phương án của mục 5. Mỗi kỹ thuật được bật hoặc tắt bằng một đối số: `hidden` (số nơ-ron của từng lớp ẩn), `batch_norm` (lớp `BatchNormalization` đặt giữa lớp tuyến tính và hàm kích hoạt), `dropout` (lớp `Dropout`), `l2` (phạt L2 qua `kernel_regularizer`) và `learning_rate`. Nhờ vậy hai phương án bất kỳ chỉ khác nhau đúng một thành phần. Ô này chỉ định nghĩa hàm, chưa in kết quả.

In [ ]:

```
def build_network(hidden=(16,), batch_norm=False, dropout=0.0,
                  l2=0.0, learning_rate=1e-3):
    """Mạng MLP 9-hidden-10; mỗi đối số bật một kỹ thuật."""
    regularizer = keras.regularizers.l2(l2) if l2 else None
    layers = [keras.layers.Input(shape=(9,))]
    for units in hidden:
        layers.append(keras.layers.Dense(
            units, activation=None if batch_norm else "relu",
            kernel_regularizer=regularizer))
        if batch_norm:      # đặt giữa lớp tuyến tính và kích hoạt
            layers += [keras.layers.BatchNormalization(),
                       keras.layers.Activation("relu")]
        if dropout:
            layers.append(keras.layers.Dropout(dropout))
    layers.append(keras.layers.Dense(10, activation="softmax"))
    model = keras.Sequential(layers)
    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate),
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"])
    return model
```

5.3. Hàm `run_variant` và `show_results`: chạy ba hạt giống và tổng hợp kết quả

Hàm `run_variant` huấn luyện một phương án với ba hạt giống, mỗi hạt giống một mạng mới dựng từ `build_network`, rồi lưu Accuracy huấn luyện, Accuracy kiểm định, số epoch và đường mất mát vào từ điển `RESULTS`. Đối số `early_stop` bật `EarlyStopping` (patience 20, khôi phục trọng số tốt nhất), `epochs` đặt số epoch tối đa (mặc định 150). Hàm này không in gì. Hàm `summarize` gom kết quả của một phương án thành một dòng gồm Accuracy huấn luyện, Accuracy kiểm định (trung bình ± độ lệch chuẩn qua ba hạt giống), độ chênh và số epoch trung bình; hàm `show_results` hiển thị các dòng đó thành bảng `pandas.DataFrame`, và mỗi phương án của mục 5 kết thúc bằng một lần gọi hàm này. Độ chênh bằng Accuracy huấn luyện trừ Accuracy kiểm định (điểm phần trăm): chênh lớn là dấu hiệu quá khớp, nhưng chỉ có ý nghĩa khi Accuracy kiểm định đã cao, vì mạng chưa khớp cũng có chênh nhỏ do cả hai Accuracy đều thấp.

In [ ]:

```
RESULTS = {}

def run_variant(label, early_stop=False, epochs=150, **options):
    """Huấn luyện ba hạt giống; đo trên tập kiểm định."""
    rows = []
    for seed in SEEDS:
        keras.utils.set_random_seed(seed)
        model = build_network(**options)
```

```
        callbacks = [keras.callbacks.EarlyStopping(
            monitor="val_loss", patience=20,
            restore_best_weights=True)] if early_stop else []
        history = model.fit(
            X_train, y_train, validation_data=(X_val, y_val),
            epochs=epochs, batch_size=32, verbose=0,
            callbacks=callbacks)
        rows.append(dict(
            train=model.evaluate(X_train, y_train, verbose=0)[1],
            val=model.evaluate(X_val, y_val, verbose=0)[1],
            epochs=len(history.history["loss"]),
            val_loss=history.history["val_loss"],
            loss=history.history["loss"]))
    RESULTS[label] = rows


def summarize(label):
    """Một dòng kết quả: trung bình qua ba hạt giống."""
    rows = RESULTS[label]
    train = [100 * r["train"] for r in rows]
    val = [100 * r["val"] for r in rows]
    return {"Phương án": label,
            "Accuracy huấn luyện (%)": round(np.mean(train), 1),
            "Accuracy kiểm định (%)":
                f"{np.mean(val):.1f} ± {np.std(val):.1f}",
            "Chênh (điểm)": round(np.mean(train) - np.mean(val), 1),
            "Số epoch": round(np.mean([r["epochs"] for r in rows]))}

def show_results(labels):
    """Hiển thị bảng kết quả của các phương án đã chạy."""
    display(pd.DataFrame([summarize(label) for label in labels]))
```

5.4. Huấn luyện lâu hơn

Chẩn đoán ở mục 4 cho thấy mạng 9–16–10 vẫn chưa học xong sau 300 epoch (Accuracy kiểm định 92,3 ± 1,0 % ở mục 4.5), nên thí nghiệm đầu tiên chỉ đổi thời gian huấn luyện: giữ nguyên mạng, cho tối đa 2000 epoch và để dừng sớm quyết định điểm dừng. Mốc so sánh là cùng mạng huấn luyện 150 epoch. Kết quả chính: Accuracy kiểm định tăng từ 86,7 ± 1,4 % lên 94,7 ± 0,4 %, mạng dừng ở trung bình 860 epoch. Kết quả được hiển thị thành bảng gồm Accuracy huấn luyện, Accuracy kiểm định (trung bình ± độ lệch chuẩn), độ chênh huấn luyện – kiểm định và số epoch trung bình.

In [ ]:

```
# Mạng cơ sở 9-16-10: 150 epoch (mốc so sánh) và huấn luyện lâu hơn
run_variant("9-16-10", hidden=(16,))
run_variant("9-16-10, huấn luyện lâu", early_stop=True, epochs=2000,
            hidden=(16,))
show_results(["9-16-10", "9-16-10, huấn luyện lâu"])
```

	Phương án	Accuracy huấn luyện (%)	Accuracy kiểm định (%)	Chênh (điểm)
0	9-16-10	89.3	86.7 ± 1.4	2.6
1	9-16-10, huấn luyện lâu	97.8	94.7 ± 0.4	3.1

5.5. Tăng kích thước mạng

Mạng mở rộng 9–256–128–10 (36.746 tham số, so với 330 tham số của mạng cơ sở) được huấn luyện 150 epoch, cùng ngân sách với mốc ở mục 5.4. Kết quả chính: Accuracy kiểm định đạt 94,8 ± 0,4 %, ngang phương án huấn luyện lâu hơn ở mục 5.4 nhưng chỉ cần 150 epoch thay vì 860. Cái giá là khoảng cách huấn luyện – kiểm định tăng từ 3,1 lên 4,6 điểm, tức mạng bắt đầu quá khớp. Kết quả được hiển thị thành bảng gồm Accuracy huấn luyện, Accuracy kiểm định (trung bình ± độ lệch chuẩn), độ chênh huấn luyện – kiểm định và số epoch trung bình.

In [ ]:

```
# Mạng mở rộng 9-256-128-10 (150 epoch)
run_variant("9-256-128-10", hidden=(256, 128))
show_results(["9-256-128-10"])
```

	Phương án	Accuracy huấn luyện (%)	Accuracy kiểm định (%)	Chênh (điểm)
0	9-256-128-10	99.4	94.8 ± 0.4	4.6

5.6. Chính quy hoá mạng mở rộng

Ba kỹ thuật chính quy hoá – L2 (hệ số 10^{-3}), Dropout (0,3) và dừng sớm – được thử riêng lẻ trên mạng 9–256–128–10, cùng với BatchNormalization (kỹ thuật ổn định huấn luyện, không thuộc nhóm chính quy hoá, đưa vào để đối chiếu), rồi gộp cả bốn với hệ số L2 giảm xuống 10^{-4}. Kết quả chính: Dropout (95,2 %) và phương án gộp (95,5 %, khoảng cách huấn luyện – kiểm định nhỏ nhất, 2,1 điểm) cao hơn mạng chưa chính quy hoá (94,8 %) từ 0,4 đến 0,7 điểm phần trăm; L2 chỉ nhỉnh hơn một chút (95,0 %), dừng sớm gần như không đổi (94,8 %) và BatchNormalization thấp hơn (94,4 %). Với ba hạt giống, chênh lệch dưới một điểm phần trăm nằm trong phạm vi dao động giữa các lần chạy nên chưa đủ để kết luận kỹ thuật nào tốt hơn; dấu hiệu rõ hơn là khoảng cách huấn luyện - kiểm định thu hẹp từ 4,6 xuống 2,1 điểm ở phương án gộp. Kết quả được hiển thị thành bảng gồm Accuracy huấn luyện, Accuracy kiểm định (trung bình ± độ lệch chuẩn), độ chênh huấn luyện – kiểm định và số epoch trung bình.

In [ ]:

```
# Mạng mở rộng 9-256-128-10, mỗi lần bật thêm MỘT kỹ thuật
run_variant("+ L2 1e-3", l2=1e-3, hidden=(256, 128))
run_variant("+ Dropout 0,3", dropout=0.3, hidden=(256, 128))
run_variant("+ BatchNormalization", batch_norm=True, hidden=(256, 128))
run_variant("+ dừng sớm", early_stop=True, hidden=(256, 128))
run_variant("+ cả bốn kỹ thuật", early_stop=True, batch_norm=True,
            dropout=0.3, l2=1e-4, hidden=(256, 128))
show_results(["+ L2 1e-3", "+ Dropout 0,3", "+ BatchNormalization",
              "+ dừng sớm", "+ cả bốn kỹ thuật"])
```

	Phương án	Accuracy huấn luyện (%)	Accuracy kiểm định (%)	Chênh (điểm)
0	+ L2 1e-3	98.9	95.0 ± 0.4	3.9
1	+ Dropout 0,3	98.8	95.2 ± 0.4	3.6
2	+ BatchNormalization	98.9	94.4 ± 0.6	4.5
3	+ dừng sớm	98.2	94.8 ± 0.4	3.4
4	+ cả bốn kỹ thuật	97.6	95.5 ± 0.6	2.1

5.7. Tổng hợp: bảng tám phương án

Ô này tổng hợp tám phương án thành một bảng `pandas.DataFrame`. Ô này gọi lại `show_results` (dùng `summarize` ở ô định nghĩa `run_variant`) cho cả tám phương án theo thứ tự thử nghiệm, với các cột Accuracy huấn luyện, Accuracy kiểm định (trung bình ± độ lệch chuẩn qua ba hạt giống), độ chênh và số epoch trung bình. Kết quả chính: hai cách chữa chưa khớp (mục 5.4 và 5.5) cho mức ngang nhau (94,7 % và 94,8 %), còn chính quy hoá (mục 5.6) thêm tối đa 0,7 điểm phần trăm và thu hẹp khoảng cách huấn luyện – kiểm định (từ 4,6 xuống 2,1 điểm ở phương án gộp). Mức tăng cùng bậc với độ lệch chuẩn nên cần nhiều hạt giống hơn để khẳng định.

In [ ]:

```
ORDER = ["9-16-10", "9-16-10, huấn luyện lâu", "9-256-128-10",
         "+ L2 1e-3", "+ Dropout 0,3", "+ BatchNormalization",
         "+ dừng sớm", "+ cả bốn kỹ thuật"]

show_results(ORDER)
```

	Phương án	Accuracy huấn luyện (%)	Accuracy kiểm định (%)	Chênh (điểm)
0	9-16-10	89.3	86.7 ± 1.4	2.6
1	9-16-10, huấn luyện lâu	97.8	94.7 ± 0.4	3.1
2	9-256-128-10	99.4	94.8 ± 0.4	4.6
3	+ L2 1e-3	98.9	95.0 ± 0.4	3.9
4	+ Dropout 0,3	98.8	95.2 ± 0.4	3.6
5	+ BatchNormalization	98.9	94.4 ± 0.6	4.5
6	+ dừng sớm	98.2	94.8 ± 0.4	3.4
7	+ cả bốn kỹ thuật	97.6	95.5 ± 0.6	2.1

6. Tinh chỉnh siêu tham số

6.1. Phương pháp làm ở mục 6: tìm kiếm siêu tham số và kiểm định chéo

Siêu tham số là những lựa chọn đặt trước khi huấn luyện và không học được từ dữ liệu, gồm số lớp ẩn, số nơ-ron mỗi lớp, tốc độ học, tỉ lệ Dropout, hệ số L2 và việc có dùng BatchNormalization hay không. Mục 5 chỉ thử một vài phương án theo chẩn đoán; mục 6 tìm có hệ thống trong cả không gian siêu tham số để chọn cấu hình tốt nhất.

Có hai cách tìm kiếm phổ biến. Tìm kiếm lưới (Grid Search) khai báo một danh sách giá trị cho từng siêu tham số rồi thử mọi tổ hợp. Cách này đầy đủ và dễ lặp lại, nhưng số tổ hợp tăng theo cấp số nhân với số siêu tham số (sáu siêu tham số, mỗi siêu tham số năm giá trị đã là hơn một vạn tổ hợp), và nó lặp đi lặp lại cùng vài giá trị của siêu tham số quan trọng trong khi vẫn tốn công cho siêu tham số ít ảnh hưởng. Tìm kiếm ngẫu nhiên (Random Search) bốc ngẫu nhiên giá trị của từng siêu tham số từ miền cho trước (dùng thang logarit cho tốc độ học và hệ số L2, vì các giá trị này trải qua nhiều bậc độ lớn) và chỉ thử một số cấu hình cố định theo ngân sách thời gian. Mỗi lần thử cho giá trị mới của mọi siêu tham số nên với cùng số lần huấn luyện, tìm kiếm ngẫu nhiên thường khám phá được nhiều hơn khi chỉ một vài siêu tham số thật sự quan trọng.

Mỗi cấu hình cần một điểm số đáng tin để so sánh. Nếu chỉ chia huấn luyện và kiểm định một lần, điểm số phụ thuộc vào đoạn kiểm định cụ thể và có thể may rủi. Kiểm định chéo (Cross-Validation) khắc phục điều này bằng cách huấn luyện và chấm điểm nhiều lần trên các cách chia khác nhau

rồi lấy trung bình; chi tiết ở phần giải thích kiểm định chéo bên dưới.

Mục này gồm bốn việc và một bảng tổng hợp. Trước hết xác định các siêu tham số và khoảng giá trị tìm kiếm. Mục 6.6 minh hoạ tìm kiếm lưới với hai siêu tham số, mỗi siêu tham số ba giá trị, tức chín cấu hình. Mục 6.7 dùng tìm kiếm ngẫu nhiên trên tất cả các siêu tham số, thử mười cấu hình. Cuối cùng, cấu hình tốt nhất của tìm kiếm ngẫu nhiên được huấn luyện lại và đánh giá trên tập kiểm tra. Mọi cấu hình đều được chấm bằng kiểm định chéo với ba hạt giống như ở mục 5, và việc tìm kiếm dùng thư viện Keras Tuner. Tập kiểm tra chỉ được dùng một lần ở cuối để thông tin từ tập này không lọt vào việc chọn cấu hình. Mục kết thúc bằng bảng tổng hợp năm phương án, từ mạng nông đến mạng tinh chỉnh siêu tham số, để thấy mỗi bước cải tiến đóng góp bao nhiêu.

6.2. Keras Tuner: thư viện tìm kiếm siêu tham số của Keras

Keras Tuner (`keras_tuner`, thường nạp với tên `kt`) là thư viện do nhóm Keras phát triển để tìm siêu tham số cho mô hình Keras; trên Colab cần cài bằng `pip install keras-tuner`. Cách dùng gồm ba phần.

Phần thứ nhất là hàm dựng mạng nhận một đối số `hp` (đối tượng `HyperParameters`). Trong hàm, mỗi siêu tham số được khai báo bằng một lệnh: `hp.Int` (số nguyên trong một khoảng), `hp.Float` (số thực trong một khoảng, thêm `sampling="log"` để lấy theo thang logarit), `hp.Choice` (một giá trị trong danh sách) và `hp.Boolean` (bật hoặc tắt). Mỗi lần thử, tuner đưa vào `hp` một bộ giá trị cụ thể và hàm trả về mạng đã biên dịch. Đối số `default` là giá trị mà siêu tham số nhận khi nó không được đưa vào tìm kiếm.

Phần thứ hai là chọn tuner: `kt.GridSearch` thử mọi tổ hợp, `kt.RandomSearch` thử các tổ hợp ngẫu nhiên. Các đối số chính gồm `objective` (đại lượng cần tối ưu và hướng tối ưu, `"max"` hoặc `"min"`), `max_trials` (số cấu hình thử tối đa), `seed` (cố định lần bốc ngẫu nhiên), `directory` và `project_name` (nơi lưu kết quả), `overwrite` (xoá kết quả cũ khi chạy lại). Phần thứ ba là gọi `tuner.search(...)` để chạy tìm kiếm rồi lấy cấu hình tốt nhất bằng `tuner.get_best_hyperparameters`.

Mặc định Keras Tuner chấm điểm mỗi cấu hình bằng một lần huấn luyện bằng `model.fit` và không có sẵn kiểm định chéo. Vì vậy bài này tạo lớp con của `kt.GridSearch` và `kt.RandomSearch`, ghi đè phương thức `run_trial` (phương thức mà tuner gọi cho mỗi cấu hình) để mỗi cấu hình được chấm bằng kiểm định chéo với ba hạt giống.

6.3. Xác định siêu tham số và khoảng giá trị tìm kiếm

Ô này nạp Keras Tuner và định nghĩa hai hàm. Hàm `layer_sizes` suy ra số nơ-ron từng lớp ẩn từ số lớp và số nơ-ron lớp đầu (mỗi lớp sau bằng một nửa lớp trước, tối thiểu 8). Hàm `build_model(hp)` khai báo sáu siêu tham số bằng `hp.Int`, `hp.Choice`, `hp.Boolean` và `hp.Float`, rồi gọi lại `build_network` của mục 5 để dựng mạng đã biên dịch. Khoảng tìm kiếm và giá trị mặc định như sau.

Siêu tham số	Khoảng tìm kiếm	Mặc định
`num_layers` (số lớp ẩn)	1, 2, 3	2

Siêu tham số	Khoảng tìm kiếm	Mặc định
`units_1` (số nơ-ron lớp ẩn đầu)	16, 32, 64, 128, 256	64
`learning_rate` (tốc độ học)	từ 10^{-4} đến 10^{-2}, thang logarit	10^{-3}
`dropout`	0, 0,1, 0,2, 0,3	0,1
`l2` (hệ số L2)	từ 10^{-6} đến 10^{-3}, thang logarit	10^{-4}
`batch_norm`	tắt, bật	tắt

Ô chỉ định nghĩa hàm, chưa in kết quả.

In [ ]:

```
!pip install -q keras-tuner

import keras_tuner as kt

def layer_sizes(num_layers, units_1):
    """Số nơ-ron từng lớp ẩn: lớp sau bằng một nửa lớp trước."""
    return tuple(max(units_1 // 2 ** k, 8)
                 for k in range(num_layers))

def build_model(hp):
    """Dựng mạng từ một bộ siêu tham số hp do Keras Tuner đưa vào."""
    num_layers = hp.Int("num_layers", 1, 3, default=2)
    units_1 = hp.Choice("units_1", [16, 32, 64, 128, 256], default=64)
    return build_network(
        hidden=layer_sizes(num_layers, units_1),
        batch_norm=hp.Boolean("batch_norm", default=False),
        dropout=hp.Choice("dropout", [0.0, 0.1, 0.2, 0.3],
                          default=0.1),
        l2=hp.Float("l2", 1e-6, 1e-3, sampling="log", default=1e-4),
        learning_rate=hp.Float("learning_rate", 1e-4, 1e-2,
                               sampling="log", default=1e-3))

SPACE_NAMES = ["num_layers", "units_1", "learning_rate",
               "dropout", "l2", "batch_norm"]
```

6.4. Kiểm định chéo k-fold và GroupKFold

Kiểm định chéo k-fold chia dữ liệu thành k phần (fold). Lần lượt mỗi phần được giữ lại làm tập kiểm định, mạng được huấn luyện trên k − 1 phần còn lại, nên k lần cho k điểm số; điểm của cấu hình là trung bình của k điểm số kèm độ lệch chuẩn. Mọi mẫu được dùng để kiểm định đúng một lần, nên điểm số ít phụ thuộc vào cách chia; cái giá là mỗi cấu hình phải huấn luyện k lần.

Với tín hiệu rung, các cửa sổ liền kề chồng lấp một nửa. Nếu chia ngẫu nhiên từng cửa sổ như `KFold` thông thường, hai cửa sổ gần như giống nhau có thể rơi vào hai phía của phép chia và làm điểm số cao giả tạo. `GroupKFold` của scikit-learn giải quyết bằng cách chia theo nhóm: các mẫu cùng một nhóm luôn nằm cùng một phía. Ở đây nhóm là chỉ số đoạn thời gian (`segment`) và k = 4 ứng với bốn đoạn đầu (ba đoạn huấn luyện và đoạn kiểm định); mỗi lần giữ lại trọn một đoạn. Lệnh `GroupKFold(n_splits=4).split(X, y, groups)` trả về bốn cặp chỉ số (huấn luyện, giữ lại). Đoạn thứ năm là tập kiểm tra, không tham gia kiểm định chéo.

Bộ chuẩn hoá `StandardScaler` được khớp lại trên phần huấn luyện của từng lần để dữ liệu giữ lại không ảnh hưởng đến mạng. Mỗi lần còn được lặp với ba hạt giống nên mỗi cấu hình tốn 4 × 3 = 12 lần huấn luyện.

6.5. Kiểm định chéo theo khối thời gian với ba hạt giống

Ô này chuẩn bị bốn phần cho kiểm định chéo (`GroupKFold`, nhóm là `segment`, chỉ lấy các đoạn 0-3) và định nghĩa cách chấm điểm. Lớp `CrossValidation` ghi đè `run_trial`: với mỗi cấu hình, vòng lặp chạy qua ba hạt giống và bốn phần; mỗi lần khớp lại `StandardScaler`, dựng mạng bằng `self.hypermodel.build(hp)`, huấn luyện tối đa 150 epoch với dừng sớm (patience 10) rồi đo Accuracy trên phần giữ lại. `run_trial` trả về Accuracy trung bình `cv_accuracy` (đại lượng mà tuner tối đa hoá) và độ lệch chuẩn `cv_std` của 12 điểm số. Hai lớp `CVGridSearch` và `CVRandomSearch` kết hợp lớp này với hai tuner của Keras Tuner. Ô còn định nghĩa hai hàm: `trial_table` gom các lần thử thành bảng, và `describe_best` in cấu hình tốt nhất cùng điểm kiểm định chéo của nó. Ô chỉ định nghĩa, chưa in kết quả.

In [ ]:

```
from sklearn.model_selection import GroupKFold

# Kiểm định chéo theo khối: mỗi phần giữ lại MỘT đoạn thời gian
is_search = segment <= 3              # huấn luyện + kiểm định
X_search, y_search = X[is_search], y[is_search]
folds = list(GroupKFold(n_splits=4).split(
    X_search, y_search, groups=segment[is_search]))

class CrossValidation:
    """Chấm mỗi cấu hình bằng kiểm định chéo, mỗi phần ba hạt giống."""
    def run_trial(self, trial, X, y, folds, **kwargs):
        hp = trial.hyperparameters
        scores = []
        for seed in SEEDS:
            for fit_index, hold_index in folds:
                fold_scaler = StandardScaler().fit(X[fit_index])
                X_fit = fold_scaler.transform(X[fit_index])
                X_hold = fold_scaler.transform(X[hold_index])
                keras.utils.set_random_seed(seed)
                model = self.hypermodel.build(hp)
                model.fit(X_fit, y[fit_index], verbose=0,
                          validation_data=(X_hold, y[hold_index]),
                          epochs=150, batch_size=32,
                          callbacks=[keras.callbacks.EarlyStopping(
                              monitor="val_loss", patience=10,
                              restore_best_weights=True)])
                scores.append(model.evaluate(
                    X_hold, y[hold_index], verbose=0)[1])
        return {"cv_accuracy": float(np.mean(scores)),
                "cv_std": float(np.std(scores))}

class CVGridSearch(CrossValidation, kt.GridSearch):
    pass

class CVRandomSearch(CrossValidation, kt.RandomSearch):
    pass

def trial_table(tuner, columns):
```

```
    """Bảng các cấu hình đã thử, xếp theo điểm giảm dần."""
    rows = []
    for trial in tuner.oracle.trials.values():
        row = {name: trial.hyperparameters.get(name)
               for name in columns}
        row["CV Accuracy (%)"] = round(
            100 * trial.metrics.get_last_value("cv_accuracy"), 1)
        row["Độ lệch chuẩn (±)"] = round(
            100 * trial.metrics.get_last_value("cv_std"), 1)
        rows.append(row)
    table = pd.DataFrame(rows).sort_values(
        "CV Accuracy (%)", ascending=False)
    return table.reset_index(drop=True)

def describe_best(tuner, names):
    """In cấu hình và kết quả tốt nhất của một lần tìm kiếm."""
    trial = tuner.oracle.get_best_trials(1)[0]
    print("Cấu hình tốt nhất:")
    for name in names:
        value = trial.hyperparameters.get(name)
        text = f"{value:.3g}" if isinstance(value, float) else value
        print(f"  {name:<14}{text}")
    print("CV Accuracy: "
          f"{100 * trial.metrics.get_last_value('cv_accuracy'):.1f} ± "
          f"{100 * trial.metrics.get_last_value('cv_std'):.1f} %")
```

6.6. Tìm kiếm lưới (Grid Search)

Ô này minh hoạ tìm kiếm lưới với hai siêu tham số, mỗi siêu tham số ba giá trị: số nơ-ron lớp ẩn đầu (32, 64, 128) và tốc độ học (10^{-3}, $3{\cdot}10^{-3}$, 10^{-2}), tức 3 × 3 = 9 cấu hình. Đối tượng `kt.HyperParameters` khai báo hai siêu tham số này bằng `Choice`; đối số `tune_new_entries=False` khiến bốn siêu tham số còn lại giữ giá trị mặc định ở bảng trên (hai lớp ẩn, Dropout 0,1, L2 10^{-4}, không BatchNormalization). `CVGridSearch.search` (với `verbose=0` để không in tiến trình từng lần thử; đổi thành `verbose=1` nếu muốn theo dõi) chấm từng cấu hình bằng kiểm định chéo với ba hạt giống (12 lần huấn luyện mỗi cấu hình, 108 lần cho cả lưới). Ô in bảng chín cấu hình xếp theo điểm giảm dần, rồi gọi `describe_best` để chỉ ra cấu hình tốt nhất (đủ sáu siêu tham số, kể cả bốn siêu tham số giữ mặc định) và điểm kiểm định chéo của nó. Kết quả chính: cấu hình tốt nhất là hai lớp ẩn 128–64 (số nơ-ron lớp đầu 128), tốc độ học 10^{-2}, Dropout 0,1, L2 10^{-4} và không BatchNormalization, đạt 95,1 ± 1,3 %. Ba cấu hình đứng đầu chỉ cách nhau 0,2 điểm phần trăm, nhỏ hơn độ lệch chuẩn, nên chưa thể coi cấu hình thứ nhất tốt hơn hẳn các cấu hình còn lại; cấu hình kém nhất là mạng nhỏ nhất với tốc độ học thấp nhất (32 nơ-ron, 10^{-3}: 92,7 %), vì mạng nhỏ học chậm nên chưa khớp trong 150 epoch.

In [ ]:

```
# Lưới 3 x 3 = 9 cấu hình: số nơ-ron lớp ẩn đầu x tốc độ học
hp_grid = kt.HyperParameters()
hp_grid.Choice("units_1", [32, 64, 128])
hp_grid.Choice("learning_rate", [1e-3, 3e-3, 1e-2])

grid_tuner = CVGridSearch(
    hypermodel=build_model,
    objective=kt.Objective("cv_accuracy", "max"),
    hyperparameters=hp_grid, tune_new_entries=False,
    max_trials=9, overwrite=True,
    directory="tuner", project_name="grid")
```

```
grid_tuner.search(X_search, y_search, folds, verbose=0)  # verbose=1: xem tiến trình

display(trial_table(grid_tuner, ["units_1", "learning_rate"]))
describe_best(grid_tuner, SPACE_NAMES)
```

	units_1	learning_rate	CV Accuracy (%)	Độ lệc
0	128	0.010	95.1	
1	128	0.001	95.0	
2	128	0.003	94.9	
3	32	0.010	94.7	
4	32	0.003	94.6	
5	64	0.003	94.6	
6	64	0.001	94.4	
7	64	0.010	94.2	
8	32	0.001	92.7	

```
Cấu hình tốt nhất:
  num_layers    2
  units_1       128
  learning_rate 0.01
  dropout       0.1
  l2            0.0001
  batch_norm    False
CV Accuracy: 95.1 ± 1.3 %
```

6.7. Tìm kiếm ngẫu nhiên (Random Search)

Ô này dùng tìm kiếm ngẫu nhiên trên cả sáu siêu tham số của hàm `build_model`, khoảng giá trị như bảng ở trên. `CVRandomSearch` bốc ngẫu nhiên mười cấu hình (`max_trials=10`, `seed` cố định để lặp lại được) và chấm từng cấu hình bằng cùng kiểm định chéo với ba hạt giống như ở mục 6.6, nên điểm số của hai phương án so sánh được với nhau. Ô in bảng mười cấu hình xếp theo điểm giảm dần, rồi gọi `describe_best` để chỉ ra cấu hình tốt nhất và điểm kiểm định chéo của nó; `get_best_hyperparameters` lấy cấu hình này để dùng ở mục sau. Kết quả chính: cấu hình tốt nhất là ba lớp ẩn 128–64–32, tốc độ học 0,0028, Dropout 0,1, L2 $3 \cdot 10^{-5}$ và không BatchNormalization, đạ 94,8 ± 2,0 %, ngang với lưới ở mục 6.6 (95,1 %) dù không gian tìm kiếm rộng hơn nhiều và chỉ thử mười cấu hình. Điểm số trải từ 67,4 % đến 94,8 %; năm cấu hình thấp nhất đều có tốc độ học dưới $4 \cdot 10^{-4}$ (bốn trong năm kèm Dropout 0,3), có thể do mạng chưa kịp khớp trong tối đa 150 epoch.

In [ ]:

```
# Tìm kiếm ngẫu nhiên: cả sáu siêu tham số, 10 cấu hình
random_tuner = CVRandomSearch(
    hypermodel=build_model,
    objective=kt.Objective("cv_accuracy", "max"),
    max_trials=10, seed=0, overwrite=True,
    directory="tuner", project_name="random")
random_tuner.search(X_search, y_search, folds, verbose=0)

display(trial_table(random_tuner, SPACE_NAMES))
describe_best(random_tuner, SPACE_NAMES)
best_hp = random_tuner.get_best_hyperparameters(1)[0]
```

	num_layers	units_1	learning_rate	dropout	l2	batch_norm	CV Accuracy (%)	Độ lệc
0	3	128	0.002835	0.1	0.000033	False	94.8	
1	3	64	0.001184	0.3	0.000004	True	92.2	
2	1	32	0.001416	0.3	0.000197	False	90.7	
3	1	64	0.006151	0.0	0.000018	True	90.5	
4	1	64	0.000557	0.3	0.000146	True	88.7	
5	3	128	0.000153	0.3	0.000127	True	87.4	
6	2	32	0.000391	0.0	0.000075	False	87.0	
7	1	128	0.000123	0.3	0.000018	True	84.8	
8	3	64	0.000228	0.3	0.000321	False	78.9	
9	2	16	0.000211	0.3	0.000737	True	67.4	

```
Cấu hình tốt nhất:
  num_layers    3
  units_1       128
  learning_rate 0.00283
  dropout       0.1
  l2            3.28e-05
  batch_norm    False
CV Accuracy: 94.8 ± 2.0 %
```

6.8. Huấn luyện lại cấu hình tốt nhất

Ô này huấn luyện lại cấu hình tốt nhất của tìm kiếm ngẫu nhiên, `build_model(best_hp)`, với ba hạt giống trên toàn bộ tập huấn luyện, dùng tập kiểm định để dừng sớm (tối đa 300 epoch, patience 20, như ở mục 4). Hàm `train_final` trả về mạng đã huấn luyện và số epoch đã chạy; ô in số tham số và số epoch của từng hạt giống. `final_model` là mạng của hạt giống 16, dùng ở mục 7. Kết quả chính: mạng có 11.946 tham số (so với 330 của mạng cơ sở) và dừng sớm ở epoch 76, 84 và 100 theo từng hạt giống.

In [ ]:

```
def train_final(seed):
    keras.utils.set_random_seed(seed)
    model = build_model(best_hp)
    history = model.fit(
        X_train, y_train, validation_data=(X_val, y_val),
        epochs=300, batch_size=32, verbose=0,
        callbacks=[keras.callbacks.EarlyStopping(
            monitor="val_loss", patience=20,
            restore_best_weights=True)])
    return model, len(history.history["loss"])

final_runs = [train_final(seed) for seed in SEEDS]
final_models = [model for model, _ in final_runs]
final_model = final_models[0]                    # hạt giống 16
print("Số tham số:", final_model.count_params())
print("Số epoch:", [epochs for _, epochs in final_runs])
```

```
Số tham số: 11946
Số epoch: [76, 84, 100]
```

6.9. Đánh giá cấu hình tốt nhất trên tập kiểm tra

Đây là lần duy nhất dùng tập kiểm tra. Hàm `test_scores` tính Accuracy (`model.evaluate`) và F1 macro (`f1_score`) trên tập kiểm tra cho từng mạng. Ô so sánh hai nhóm mạng, mỗi nhóm ba hạt giống: mạng cơ sở ở mục 4 (`runs` ở mục 4.5) và mạng cấu hình tốt nhất vừa huấn luyện lại, báo

cáo bằng trung bình ± độ lệch chuẩn. Kết quả chính: trên tập kiểm tra, mạng cơ sở đạt Accuracy 94,7 ± 1,7 % và F1 macro 94,5 ± 1,9 %; cấu hình cuối đạt 96,5 ± 0,6 % và 96,5 ± 0,5 %, tăng khoảng 1,8 điểm phần trăm. Mức tăng này cùng bậc với độ lệch chuẩn của mạng cơ sở qua ba hạt giống nên chỉ nên xem là xu hướng.

In [ ]:

```
# Lần DUY NHẤT dùng tập kiểm tra: mạng cơ sở và cấu hình cuối
baseline_models = [model for model, _ in runs]    # từ mục 4

def test_scores(models):
    accuracy = [m.evaluate(X_test, y_test, verbose=0)[1]
                for m in models]
    macro_f1 = []
    for m in models:
        y_pred = m.predict(X_test, verbose=0).argmax(axis=1)
        macro_f1.append(f1_score(y_test, y_pred,
                                 average="macro"))
    return accuracy, macro_f1

for name, models in (("Mạng cơ sở", baseline_models),
                     ("Cấu hình cuối", final_models)):
    accuracy, macro_f1 = test_scores(models)
    print(f"{name:<14} Accuracy "
          f"{100 * statistics.mean(accuracy):.1f} ± "
          f"{100 * statistics.pstdev(accuracy):.1f} % | F1 macro "
          f"{100 * statistics.mean(macro_f1):.1f} ± "
          f"{100 * statistics.pstdev(macro_f1):.1f} %")
```

```
Mạng cơ sở     Accuracy 94.7 ± 1.7 % | F1 macro 94.5 ± 1.9 %
Cấu hình cuối  Accuracy 96.5 ± 0.6 % | F1 macro 96.5 ± 0.5 %
```

6.10. Tổng hợp: bảng năm phương án

Ô này tổng hợp năm phương án thành một bảng cùng định dạng với bảng cuối mục 5, theo trình tự cải tiến: mạng nông 9–16–10 (150 epoch), mạng nông huấn luyện lâu (dừng sớm, tối đa 2000 epoch), mạng sâu 9–256–128–10 không chính quy hoá, mạng sâu chính quy hoá cả bốn kỹ thuật (L2, Dropout, dừng sớm và BatchNormalization) và mạng tinh chỉnh siêu tham số tốt nhất. Bốn phương án đầu lấy từ `RESULTS` của mục 5; kết quả ba hạt giống của mạng tinh chỉnh (huấn luyện lại ở trên) được thêm vào `RESULTS` cùng định dạng rồi gom bằng `summarize`. Bảng gồm kiến trúc, Accuracy huấn luyện, Accuracy kiểm định (trung bình ± độ lệch chuẩn), độ chênh huấn luyện – kiểm định và số epoch trung bình; mọi số liệu trong bảng đo trên tập huấn luyện và tập kiểm định, còn kết quả trên tập kiểm tra nằm ở ô đánh giá phía trên. Kết quả chính: Accuracy kiểm định tăng từ 86,7 % ở mạng nông lên 94,7 % khi huấn luyện lâu hơn và 94,8 % khi mở rộng mạng; chính quy hoá cả bốn kỹ thuật đạt 95,5 % với độ chênh nhỏ hơn (2,1 điểm); mạng tinh chỉnh siêu tham số cao nhất, 95,9 % (độ lệch chuẩn gần bằng 0), chỉ với 11.946 tham số (so với 36.746 của mạng 9–256–128–10) và dừng sau 87 epoch. Số liệu kiểm định của mạng tinh chỉnh có thể hơi lạc quan, vì đoạn kiểm định nằm trong dữ liệu dùng để chọn siêu tham số (ở lần chạy này Accuracy kiểm tra còn cao hơn một chút); con số dùng để kết luận là Accuracy trên tập kiểm tra ở ô đánh giá phía trên (96,5 ± 0,6 %).

In [ ]:

```
RESULTS["Tinh chỉnh siêu tham số"] = [
    dict(train=model.evaluate(X_train, y_train, verbose=0)[1],
         val=model.evaluate(X_val, y_val, verbose=0)[1],
         epochs=epochs)
    for model, epochs in final_runs]

best_sizes = layer_sizes(best_hp.get("num_layers"),
                         best_hp.get("units_1"))
FINAL_VARIANTS = [   # (khoá trong RESULTS, tên phương án, kiến trúc)
    ("9-16-10", "Mạng nông", "9-16-10"),
    ("9-16-10, huấn luyện lâu", "Mạng nông, huấn luyện lâu",
     "9-16-10"),
    ("9-256-128-10", "Mạng sâu, không chính quy hoá",
     "9-256-128-10"),
    ("+ cả bốn kỹ thuật", "Mạng sâu, chính quy hoá cả bốn kỹ thuật",
     "9-256-128-10"),
    ("Tinh chỉnh siêu tham số", "Mạng tinh chỉnh siêu tham số tốt nhất",
     "9-" + "-".join(map(str, best_sizes)) + "-10")]

final_table = pd.DataFrame([summarize(key)
                            for key, _, _ in FINAL_VARIANTS])
final_table["Phương án"] = [name for _, name, _ in FINAL_VARIANTS]
final_table.insert(1, "Kiến trúc",
                   [arch for _, _, arch in FINAL_VARIANTS])
display(final_table)
```

	Phương án	Kiến trúc	Accuracy huấn luyện (%)	Accuracy kiểm định (%)	Chênl (điểm
0	Mạng nông	9-16-10	89.3	86.7 ± 1.4	2.(
1	Mạng nông, huấn luyện lâu	9-16-10	97.8	94.7 ± 0.4	3.
2	Mạng sâu, không chính quy hoá	9-256-128-10	99.4	94.8 ± 0.4	4.(
3	Mạng sâu, chính quy hoá cả bốn kỹ thuật	9-256-128-10	97.6	95.5 ± 0.6	2.
4	Mạng tinh chỉnh siêu tham số tốt nhất	9-128-64-32-10	98.9	95.9 ± 0.0	3.(

7. Lưu và dùng lại mô hình

7.1. Phương pháp làm ở mục 7: lưu mô hình và dùng lại

Sau khi chọn được cấu hình tốt nhất, bước cuối cùng là lưu mô hình để dùng lại mà không phải huấn luyện lại. Mạng chỉ dùng được khi đi cùng cách xử lý dữ liệu đã dùng lúc huấn luyện, nên phải lưu cả hai: mạng `final_model` (tệp `.keras` chứa kiến trúc, trọng số và cấu hình biên dịch) và các thành phần tiền xử lý gồm bộ chuẩn hoá `scaler`, danh sách tên lớp, độ dài cửa sổ và bước trượt (lưu bằng `joblib`). Cả hai được lưu vào một thư mục trên Google Drive để còn nguyên sau khi phiên Colab kết thúc.

Khi dùng lại, ta nạp mạng bằng `keras.models.load_model` và tiền xử lý bằng `joblib.load`, kiểm tra mạng nạp lại cho dự đoán trùng với mạng gốc, rồi chẩn đoán trực tiếp từ một cửa sổ tín hiệu rung thô: trích chín đặc trưng, chuẩn hoá bằng bộ chuẩn hoá đã lưu rồi đưa vào mạng. Tệp lưu không chứa mã của hàm trích đặc trưng `extract_features`; trong một notebook mới phải khai bác lại hàm này đúng như ở mục 3, nếu không đặc trưng sẽ khác lúc huấn luyện và dự đoán sẽ sai.

7.2. Lưu mô hình

Ô này khai báo thư mục `MODEL_DIR` trên Drive (Drive đã được kết nối ở mục 2), tạo thư mục nếu chưa có, rồi lưu hai tệp. Tệp `bearing_mlp.keras` là mạng của hạt giống 16 trong cấu hình tốt nhất, lưu bằng `final_model.save`. Tệp `bearing_preprocess.joblib` là một từ điển gồm `scaler`, `class_names`, `WINDOW_SIZE` và `STRIDE`, lưu bằng `joblib.dump`. Ô in danh sách tệp trong thư mục để xác nhận đã lưu.

In [ ]:

```
import os
import joblib

MODEL_DIR = "/content/gdrive/MyDrive/Colab Notebooks/Model"  # thư mục trên Drive
os.makedirs(MODEL_DIR, exist_ok=True)

# Lưu CẢ HAI: mạng và các thành phần tiền xử lý
final_model.save(f"{MODEL_DIR}/bearing_mlp.keras")
joblib.dump({"scaler": scaler, "class_names": class_names,
             "window_size": WINDOW_SIZE, "stride": STRIDE},
            f"{MODEL_DIR}/bearing_preprocess.joblib")

print(sorted(os.listdir(MODEL_DIR)))
```

```
['bearing_mlp.keras', 'bearing_preprocess.joblib']
```

7.3. Gọi mô hình đã lưu từ Drive để dùng lại

Phần này mô phỏng việc dùng lại mô hình sau khi huấn luyện xong. Trong một phiên mới chỉ cần kết nối Drive, nạp `keras`, `numpy` và `joblib`, khai báo lại hàm `extract_features` rồi chạy hai ô dưới đây; không cần dữ liệu huấn luyện hay việc huấn luyện lại.

7.3.1. Nạp mô hình và kiểm tra dự đoán trùng khớp

Ô này nạp mạng bằng `keras.models.load_model` và các thành phần tiền xử lý bằng `joblib.load` từ `MODEL_DIR`. Để chắc chắn việc lưu và nạp không làm đổi mô hình, ô cho cả mạng nạp lại và mạng gốc dự đoán trên tập kiểm tra rồi so sánh bằng `np.allclose`; kết quả `True` nghĩa là hai mạng cho dự đoán trùng nhau. Kết quả: `True`.

In [ ]:

```
# Nạp lại mạng và tiền xử lý đã lưu trên Drive
loaded_model = keras.models.load_model(f"{MODEL_DIR}/bearing_mlp.keras")
preprocess = joblib.load(f"{MODEL_DIR}/bearing_preprocess.joblib")

# Mạng nạp lại phải cho dự đoán trùng mạng gốc
same = np.allclose(loaded_model.predict(X_test, verbose=0),
                   final_model.predict(X_test, verbose=0))
print("Dự đoán của mạng nạp lại trùng mạng gốc:", same)
```

```
Dự đoán của mạng nạp lại trùng mạng gốc: True
```

7.3.2. Chẩn đoán từ cửa sổ tín hiệu rung thô, mỗi trạng thái một cửa sổ

Hàm `diagnose_window` nhận một cửa sổ tín hiệu thô dài `WINDOW_SIZE` điểm và trả về trạng thái ổ lăn cùng độ tin cậy: trích chín đặc trưng bằng `extract_features`, chuẩn hoá bằng `preprocess["scaler"]`, đưa vào `loaded_model` rồi lấy lớp có xác suất lớn nhất (`argmax`) và xác suất của nó. Ô gọi hàm này cho mười cửa sổ, mỗi trạng thái một cửa sổ lấy từ đoạn kiểm tra của tín hiệu tương ứng (cùng vị trí `start`), rồi gom kết quả vào một bảng `pandas` gồm trạng thái thật, trạng thái dự đoán, độ tin cậy và cột `Đúng`. Mỗi trạng thái chỉ thử một cửa sổ nên bảng chỉ minh hoạ cách gọi mô hình, không thay cho việc đánh giá trên cả tập kiểm tra ở mục 6.9. Kết quả: 8 trong 10 cửa sổ được dự đoán đúng; hai cửa sổ sai (con lăn 0,007 in và vòng ngoài 0,014 in) đều bị nhận nhầm thành con lăn 0,021 in. Cửa sổ vòng ngoài 0,014 in có độ tin cậy thấp (0,48), nhưng cửa sổ con lăn 0,007 in sai với độ tin cậy 0,85, cao hơn cả cửa sổ đúng thấp nhất (con lăn 0,021 in, 0,71); như vậy độ tin cậy thấp là dấu hiệu cần kiểm tra lại, còn độ tin cậy cao chưa bảo đảm kết quả đúng.

In [ ]:

```
def diagnose_window(raw_window):
    """Dạng sóng thô 2048 điểm -> (trạng thái, độ tin cậy)."""
    features = np.asarray([extract_features(raw_window)])
    features = preprocess["scaler"].transform(features)
    proba = loaded_model.predict(features, verbose=0)[0]
    best = int(proba.argmax())
    return preprocess["class_names"][best], float(proba[best])

# Mỗi trạng thái một cửa sổ, lấy từ đoạn kiểm tra của tín hiệu
start = int(0.8 * min_length) + 3 * STRIDE
rows = []
for state in preprocess["class_names"]:
    raw = signals[state][:min_length]
    window = raw[start:start + preprocess["window_size"]]
    predicted, confidence = diagnose_window(window)
    rows.append({"Trạng thái thật": state, "Dự đoán": predicted,
                 "Độ tin cậy": round(confidence, 2),
                 "Đúng": predicted == state})
display(pd.DataFrame(rows))
```

	Trạng thái thật	Dự đoán	Độ tin cậy
0	Bình thường	Bình thường	1.00
1	Vòng trong 0,007in	Vòng trong 0,007in	1.00
2	Vòng trong 0,014in	Vòng trong 0,014in	1.00
3	Vòng trong 0,021in	Vòng trong 0,021in	1.00
4	Con lăn 0,007in	Con lăn 0,021in	0.85
5	Con lăn 0,014in	Con lăn 0,014in	1.00
6	Con lăn 0,021in	Con lăn 0,021in	0.71
7	Vòng ngoài 0,007in	Vòng ngoài 0,007in	1.00
8	Vòng ngoài 0,014in	Con lăn 0,021in	0.48
9	Vòng ngoài 0,021in	Vòng ngoài 0,021in	1.00

8. Tổng kết quy trình

Quy trình sáu bước được áp dụng liên tiếp cho bài toán chẩn đoán trạng thái ổ lăn bằng mạng MLP.

Mục	Việc đã làm	Đầu ra chính
2. Xác định bài toán, dữ liệu	Đọc mười tệp CWRU, quan sát dạng sóng, xác định bài toán phân loại 10 trạng thái	Bài toán và dữ liệu rõ ràng
3. Chia tập và tiền xử lý	Trích chín đặc trưng cho mỗi cửa sổ, chia ba tập theo đoạn thời gian, chuẩn hoá chỉ trên tập huấn luyện	Ba tập không rò rỉ thông tin và bộ chuẩn hoá `scaler`
4. Khai báo và huấn luyện mạng cơ bản	Dựng mạng 9–16–10, khai báo Adam, mất mát `sparse_categorical_crossentropy`, Accuracy và dừng sớm; huấn luyện với một hạt giống rồi ba hạt giống, vẽ đường học và ma trận nhầm lẫn	Mạng `model_bs`, mốc so sánh, biết mạng chưa khớp hay quá khớp và nhầm lẫn kiểu nào
5. Cải tiến theo thử và sai	Huấn luyện lâu hơn, tăng kích thước mạng, thêm kỹ thuật chính quy hoá	Bảng so sánh tám phương án
6. Tinh chỉnh siêu tham số	Tìm kiếm lưới và tìm kiếm ngẫu nhiên với kiểm định chéo, huấn luyện lại, đánh giá một lần trên tập kiểm tra	Cấu hình tốt nhất và bảng so sánh năm phương án
7. Lưu và dùng lại mô hình	Lưu mạng cùng tiền xử lý, nạp lại từ Drive để chẩn đoán	Mô hình dùng lại được mà không cần huấn luyện lại

Quy trình lặp lại một vòng: huấn luyện, chẩn đoán, thay đổi một yếu tố, đo lại. Bốn nguyên tắc đi xuyên suốt cả quy trình. Dữ liệu chia theo đoạn thời gian và bộ chuẩn hoá chỉ khớp trên tập huấn luyện để không rò rỉ thông tin. Mỗi mạng được huấn luyện với nhiều hạt giống và báo cáo bằng trung bình ± độ lệch chuẩn, vì chênh lệch nhỏ có thể chỉ do khởi tạo ngẫu nhiên. Mỗi lần chỉ đổi một yếu tố để biết yếu tố nào có tác dụng. Tập kiểm tra chỉ được dùng một lần ở cuối, và khi lưu mô hình phải lưu cả mạng lẫn tiền xử lý.